

Lab Manuals

1. Lab Title

Implementation of Singly Linked List and Basic Operations in C++

2. Lab Objectives

After completing this lab, students will be able to:

- understand the structure of a singly linked list
- create nodes dynamically
- insert nodes at start, end, and specific position
- update nodes at different positions
- delete nodes from start, end, and specific position
- analyze time complexity of each operation
- optimize traversal where possible

3. Introduction / Theory

A **singly linked list** is a linear dynamic data structure in which each node contains:

1. **data**
2. **pointer to next node**

Unlike arrays, linked lists do not require contiguous memory locations. Nodes are created dynamically and linked through pointers.

Structure of a Singly Linked List Node

```
struct Node {  
    int data;  
    Node* next;  
};
```

Example

[10 | *] -> [20 | *] -> [30 | NULL]

Here:

- 10, 20, 30 are data values

- each node points to the next node
- the last node points to NULL

4. Why Linked List?

Linked lists are useful when:

- size is not fixed
- insertion/deletion occurs frequently
- memory needs to be allocated dynamically

5. Complete Program for Singly Linked List

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class SinglyLinkedList {
private:
    Node* head;

public:
    SinglyLinkedList() {
        head = NULL;
    }

    // Display list
    void display() {
        if (head == NULL) {
            cout << "List is empty.\n";
            return;
        }
    }
}
```

```
Node* temp = head;
while (temp != NULL) {
    cout << temp->data << " -> ";
    temp = temp->next;
}
cout << "NULL\n";
}
```

// 1. Add node at the start

```
void addAtStart(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}
```

// 2. Add node at the end

```
void addAtEnd(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }
```

```
Node* temp = head;
while (temp->next != NULL) {
    temp = temp->next;
}

temp->next = newNode;
}
```

// 3. Add node at specific position

```
void addAtPosition(int value, int position) {
```

```

if (position < 1) {
    cout << "Invalid position.\n";
    return;
}

if (position == 1) {
    addAtStart(value);
    return;
}

Node* newNode = new Node();
newNode->data = value;
newNode->next = NULL;

Node* temp = head;
for (int i = 1; i < position - 1 && temp != NULL; i++) {
    temp = temp->next;
}

if (temp == NULL) {
    cout << "Position out of range.\n";
    delete newNode;
    return;
}

newNode->next = temp->next;
temp->next = newNode;
}

// 4. Update first node
void updateFirstNode(int value) {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }
    head->data = value;
}

```

// 5. Update last node

```
void updateLastNode(int value) {  
    if (head == NULL) {  
        cout << "List is empty.\n";  
        return;  
    }  

```

```
    Node* temp = head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  

```

```
    temp->data = value;  
}
```

// 6. Update node at specific position

```
void updateAtPosition(int value, int position) {  
    if (position < 1) {  
        cout << "Invalid position.\n";  
        return;  
    }  

```

```
    Node* temp = head;  
    for (int i = 1; i < position && temp != NULL; i++) {  
        temp = temp->next;  
    }  

```

```
    if (temp == NULL) {  
        cout << "Position out of range.\n";  
        return;  
    }  

```

```
    temp->data = value;  
}
```

// 7. Delete first node

```
void deleteFirstNode() {  
    if (head == NULL) {
```

```

        cout << "List is empty.\n";
        return;
    }

    Node* temp = head;
    head = head->next;
    delete temp;
}

// 8. Delete last node
void deleteLastNode() {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }

    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    Node* temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = NULL;
}

// 9. Delete node at specific position
void deleteAtPosition(int position) {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }

```

```
    if (position < 1) {
        cout << "Invalid position.\n";
        return;
    }

    if (position == 1) {
        deleteFirstNode();
        return;
    }

    Node* temp = head;
    for (int i = 1; i < position - 1 && temp->next != NULL; i++) {
        temp = temp->next;
    }

    if (temp->next == NULL) {
        cout << "Position out of range.\n";
        return;
    }

    Node* nodeToDelete = temp->next;
    temp->next = nodeToDelete->next;
    delete nodeToDelete;
}

};
```

```
int main() {
    SinglyLinkedList list;

    list.addAtEnd(10);
    list.addAtEnd(20);
    list.addAtEnd(30);
    cout << "Initial List: ";
    list.display();

    list.addAtStart(5);
    cout << "After adding at start: ";
    list.display();
}
```

```
list.addAtPosition(15, 3);
cout << "After adding 15 at position 3: ";
list.display();

list.updateFirstNode(100);
cout << "After updating first node: ";
list.display();

list.updateLastNode(300);
cout << "After updating last node: ";
list.display();

list.updateAtPosition(200, 3);
cout << "After updating position 3: ";
list.display();

list.deleteFirstNode();
cout << "After deleting first node: ";
list.display();

list.deleteLastNode();
cout << "After deleting last node: ";
list.display();

list.deleteAtPosition(2);
cout << "After deleting position 2: ";
list.display();

return 0;
}
```

6. Explanation of Operations

1. Add Node at the End

A new node is created. Traverse the list until the last node and attach the new node there.

2. Add Node at Specific Position

Traverse to the node just before the target position, then adjust links.

3. Add Node at the Start

Create a new node and make it point to the current head. Then move head to the new node.

4. Update the Last Node

Traverse to the last node and replace its value.

5. Update the First Node

Directly modify head->data.

6. Update Node at Specific Position

Traverse to the required position and update node value.

7. Delete the First Node

Move head to next node and delete old first node.

8. Delete the Last Node

Traverse to second last node, delete last node, and set second last node's next to NULL.

9. Delete Node at Specific Position

Traverse to previous node, bypass target node, then delete it.

7. Sample Output

Initial List: 10 -> 20 -> 30 -> NULL

After adding at start: 5 -> 10 -> 20 -> 30 -> NULL

After adding 15 at position 3: 5 -> 10 -> 15 -> 20 -> 30 -> NULL

After updating first node: 100 -> 10 -> 15 -> 20 -> 30 -> NULL

After updating last node: 100 -> 10 -> 15 -> 20 -> 300 -> NULL

After updating position 3: 100 -> 10 -> 200 -> 20 -> 300 -> NULL

After deleting first node: 10 -> 200 -> 20 -> 300 -> NULL

After deleting last node: 10 -> 200 -> 20 -> NULL

After deleting position 2: 10 -> 20 -> NULL

8. Time Complexity Table for Singly Linked List

Operation	Time Complexity
Add at start	$O(1)$
Add at end	$O(n)$
Add at position	$O(n)$
Update first	$O(1)$
Update last	$O(n)$
Update at position	$O(n)$
Delete first	$O(1)$
Delete last	$O(n)$
Delete at position	$O(n)$

9. Optimization of Singly Linked List

Possible Optimization

The main costly operation is traversing to the end repeatedly.

Improvement:

Maintain an additional pointer:

Node* tail;

With tail, these operations improve:

- add at end becomes **$O(1)$**
- update last becomes **$O(1)$**

However:

- delete last still remains **$O(n)$** in singly linked list because we must find the previous node

Further Possible Enhancements

- maintain node count
- validate position quickly
- use tail pointer
- modularize repeated traversal code

10. Viva / Conceptual Questions

1. Why is insertion at start $O(1)$ in linked list?
2. Why does deletion at end take $O(n)$ in singly linked list?
3. What is the role of head?
4. Why does linked list not need contiguous memory?
5. What happens if a pointer is not updated correctly?

1. Lab Manual: Doubly Linked List

Implementation of Doubly Linked List and Basic Operations in C++

2. Lab Objectives

After completing this lab, students will be able to:

- understand how doubly linked list works
- differentiate between singly and doubly linked list
- implement insertion, update, and deletion operations
- use both next and prev pointers correctly
- understand benefits and drawbacks of doubly linked list

3. Introduction / Theory

A **doubly linked list** is a linear dynamic data structure in which each node contains:

1. **data**
2. **pointer to next node**
3. **pointer to previous node**

Structure

```
struct Node {  
    int data;  
    Node* prev;  
    Node* next;  
};
```

Example

NULL <- [10] <-> [20] <-> [30] -> NULL

How it Works

Each node is connected in both directions:

- next points to next node
- prev points to previous node

This makes traversal possible:

- forward
- backward

Advantages over Singly Linked List

- easier backward traversal
- easier deletion of a known node
- convenient for browser history, undo/redo, playlists

Disadvantages

- extra memory for prev
- more pointer updates during insertion and deletion

4. Complete Program for Doubly Linked List

```
#include <iostream>
using namespace std;
```

```
struct DNode {
    int data;
    DNode* prev;
    DNode* next;
};
```

```
class DoublyLinkedList {
private:
    DNode* head;
```

```
public:
    DoublyLinkedList() {
        head = NULL;
    }
```

```
// Display forward
void displayForward() {
```

```

if (head == NULL) {
    cout << "List is empty.\n";
    return;
}

DNode* temp = head;
while (temp != NULL) {
    cout << temp->data << " <-> ";
    temp = temp->next;
}
cout << "NULL\n";
}

// 1. Add node at start
void addAtStart(int value) {
    DNode* newNode = new DNode();
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL) {
        head->prev = newNode;
    }

    head = newNode;
}

// 2. Add node at end
void addAtEnd(int value) {
    DNode* newNode = new DNode();
    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }

```

```

DNode* temp = head;
while (temp->next != NULL) {
    temp = temp->next;
}

temp->next = newNode;
newNode->prev = temp;
}

// 3. Add node at specific position
void addAtPosition(int value, int position) {
    if (position < 1) {
        cout << "Invalid position.\n";
        return;
    }

    if (position == 1) {
        addAtStart(value);
        return;
    }

    DNode* temp = head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        cout << "Position out of range.\n";
        return;
    }

    DNode* newNode = new DNode();
    newNode->data = value;
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL) {

```

```

        temp->next->prev = newNode;
    }

    temp->next = newNode;
}

// 4. Update first node
void updateFirstNode(int value) {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }
    head->data = value;
}

// 5. Update last node
void updateLastNode(int value) {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }

    DNode* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->data = value;
}

// 6. Update node at specific position
void updateAtPosition(int value, int position) {
    if (position < 1) {
        cout << "Invalid position.\n";
        return;
    }

    DNode* temp = head;

```

```
for (int i = 1; i < position && temp != NULL; i++) {  
    temp = temp->next;  
}  
  
if (temp == NULL) {  
    cout << "Position out of range.\n";  
    return;  
}  
  
temp->data = value;  
}
```

```
// 7. Delete first node  
void deleteFirstNode() {  
    if (head == NULL) {  
        cout << "List is empty.\n";  
        return;  
    }  
}
```

```
    DNode* temp = head;  
    head = head->next;  
  
    if (head != NULL) {  
        head->prev = NULL;  
    }  
  
    delete temp;  
}
```

```
// 8. Delete last node  
void deleteLastNode() {  
    if (head == NULL) {  
        cout << "List is empty.\n";  
        return;  
    }  
}
```

```
    if (head->next == NULL) {  
        delete head;
```

```

    head = NULL;
    return;
}

DNode* temp = head;
while (temp->next != NULL) {
    temp = temp->next;
}

temp->prev->next = NULL;
delete temp;
}

// 9. Delete node at specific position
void deleteAtPosition(int position) {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }

    if (position < 1) {
        cout << "Invalid position.\n";
        return;
    }

    if (position == 1) {
        deleteFirstNode();
        return;
    }

    DNode* temp = head;
    for (int i = 1; i < position && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        cout << "Position out of range.\n";
        return;
    }

```

```

    }

    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }

    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    }

    delete temp;
}
};

int main() {
    DoublyLinkedList list;

    list.addAtEnd(10);
    list.addAtEnd(20);
    list.addAtEnd(30);
    cout << "Initial Doubly Linked List: ";
    list.displayForward();

    list.addAtStart(5);
    cout << "After adding at start: ";
    list.displayForward();

    list.addAtPosition(15, 3);
    cout << "After adding 15 at position 3: ";
    list.displayForward();

    list.updateFirstNode(100);
    cout << "After updating first node: ";
    list.displayForward();

    list.updateLastNode(300);
    cout << "After updating last node: ";
    list.displayForward();
}

```

```

list.updateAtPosition(200, 3);
cout << "After updating node at position 3: ";
list.displayForward();

list.deleteFirstNode();
cout << "After deleting first node: ";
list.displayForward();

list.deleteLastNode();
cout << "After deleting last node: ";
list.displayForward();

list.deleteAtPosition(2);
cout << "After deleting node at position 2: ";
list.displayForward();

return 0;
}

```

5. Explanation of Doubly Linked List Operations

1. Add at Start

- create new node
- set newNode->next = head
- set old head's prev to new node
- move head to new node

2. Add at End

- traverse to last node
- connect last node's next to new node
- set new node's prev to last node

3. Add at Specific Position

- move to the node before target position

- adjust four links carefully:
 - previous node next
 - new node prev
 - new node next
 - next node prev

4. Update First Node

Directly modify head->data

5. Update Last Node

Traverse to last node and change value

6. Update Node at Specific Position

Traverse to required node and update data

7. Delete First Node

Move head to next node and set its prev = NULL

8. Delete Last Node

Traverse to last node, then connect previous node's next = NULL

9. Delete at Specific Position

Reconnect previous and next nodes around target node, then delete it

6. Time Complexity Table for Doubly Linked List

Operation	Time Complexity
Add at start	$O(1)$
Add at end	$O(n)$
Add at position	$O(n)$
Update first	$O(1)$

Operation	Time Complexity
-----------	-----------------

Update last	$O(n)$
-------------	--------

Update at position	$O(n)$
--------------------	--------

Delete first	$O(1)$
--------------	--------

Delete last	$O(n)$
-------------	--------

Delete at position	$O(n)$
--------------------	--------

7. Optimization for Doubly Linked List

Use Both Head and Tail

If we maintain:

DNode* head;

DNode* tail;

Then:

- add at end becomes **$O(1)$**
- update last becomes **$O(1)$**
- delete last becomes **$O(1)$**

This is one of the major advantages of doubly linked list over singly linked list.

Further Improvements

- maintain node count
- use helper traversal methods
- choose traversal direction based on position:
 - from head if position is in first half
 - from tail if position is in second half

That can reduce practical traversal time.

8. Singly vs Doubly Linked List

Feature	Singly Linked List	Doubly Linked List
Pointers per node	1	2
Forward traversal	Yes	Yes
Backward traversal	No	Yes
Memory usage	Less	More
Deletion of known node	Harder	Easier
Implementation	Simpler	More complex

9. Sample Output

Initial Doubly Linked List: 10 <-> 20 <-> 30 <-> NULL

After adding at start: 5 <-> 10 <-> 20 <-> 30 <-> NULL

After adding 15 at position 3: 5 <-> 10 <-> 15 <-> 20 <-> 30 <-> NULL

After updating first node: 100 <-> 10 <-> 15 <-> 20 <-> 30 <-> NULL

After updating last node: 100 <-> 10 <-> 15 <-> 20 <-> 300 <-> NULL

After updating node at position 3: 100 <-> 10 <-> 200 <-> 20 <-> 300 <-> NULL

After deleting first node: 10 <-> 200 <-> 20 <-> 300 <-> NULL

After deleting last node: 10 <-> 200 <-> 20 <-> NULL

After deleting node at position 2: 10 <-> 20 <-> NULL

10. Discussion / Conceptual Questions

1. Why does doubly linked list use more memory?
2. Why is deletion easier in doubly linked list?
3. Why do we need both prev and next?
4. Which linked list is better for browser history and why?
5. Which operations become faster if we maintain tail?

Task A: Singly Linked List

Implement the following:

1. add node at end
2. add node at specific position
3. add node at start
4. update last node
5. update first node
6. update node at specific position
7. delete first node
8. delete last node
9. delete node at specific position
10. optimize using tail/count if possible

Task B: Doubly Linked List

1. explain how doubly linked list works
2. add node at end
3. add node at specific position
4. add node at start
5. update last node
6. update first node
7. update node at specific position
8. delete first node
9. delete last node
10. delete node at specific position
11. optimize using head + tail if possible

Students should submit a **single PDF or Word document** containing:

1. Lab Title
2. Objectives
3. Algorithms
4. Source Code
5. Different Practice Variations Code Screenshots
6. Output Screenshots
7. Dry Run Explanation
8. Time Complexity
9. Optimization Notes
10. Conclusion
11. Viva Questions - Answers